

Unidad 12: Despliegue y CI/CD
INFORME POST-1
Docker y CI/CD con GitHub Actions

Alejandra Farfán Duarte
02230131026

Jonathan Rolando Rey Castillo

Universidad De Santander
Programación Web
Ingeniería De Sistemas
Cúcuta, Norte De Santander
2026

1. Introducción

En este laboratorio se contenedorizó una aplicación Spring Boot mediante un Dockerfile multi-stage, se configuró el ambiente de producción con perfiles y variables de entorno, se orquestaron los servicios localmente con Docker Compose incluyendo PostgreSQL, y se desplegó la aplicación en Railway con la base de datos conectada y los endpoints REST funcionando públicamente.

2. Parte 1: Dockerfile Multi-Stage

Se creó el archivo Dockerfile en la raíz del proyecto con dos etapas:

Etapla builder: usa la imagen eclipse-temurin:21-jdk-alpine con Maven instalado para compilar el proyecto y generar el JAR.

Etapla producción: usa la imagen eclipse-temurin:21-jre-alpine más liviana, con un usuario no root por seguridad, copiando únicamente el JAR compilado.

También se creó el archivo .dockerignore para excluir target/ y .git/ del contexto de construcción.

Figura 1. Dockerfile multi-stage en la raíz del proyecto

3. Parte 2: Perfiles y Docker Compose

Se configuraron dos perfiles de Spring Boot:

application.properties — perfil por defecto, conecta a PostgreSQL via variables de entorno

application-prod.properties — perfil de producción para Railway, con logs reducidos y SQL oculto

Se creó el archivo docker-compose.yml orquestando dos servicios:

app — la aplicación Spring Boot en el puerto 8080

db — PostgreSQL 16 con healthcheck configurado

Figura 2. Resultado de docker compose ps mostrando ambos contenedores activos

Figura 3. Health check local en <http://localhost:8080/actuator/health> con status UP

4. Parte 3: Despliegue en Railway

Se conectó el repositorio de GitHub a Railway, que detectó el Dockerfile automáticamente. Se aprovisionó una base de datos PostgreSQL y se configuraron las variables de entorno:

```
VariableValorSPRING_PROFILES_ACTIVEprodDATABASE_URLjdbc:postgresql://...DB_USER${Postgres.PGUSER}DB_PASS${Postgres.PGPASSWORD}
```

Se generó el dominio público y se verificó el funcionamiento de la aplicación.

Figura 4. Panel de Railway con los servicios app y PostgreSQL activos

Figura 5. Variables de entorno configuradas en Railway

Figura 6. Deployment exitoso en Railway

Figura 7. Health check en producción con status UP y PostgreSQL conectado

Figura 8. Endpoint POST /api/productos — creación de producto

Figura 9. Endpoint GET /api/productos — listado de productos

Figura 10. Endpoint GET /api/productos/1 — búsqueda por ID

5. Conclusiones

Se logró contenedorizar exitosamente la aplicación Spring Boot con un Dockerfile multi-stage, reduciendo el tamaño de la imagen al usar solo JRE en producción. La orquestación local con Docker Compose permitió probar el stack completo antes del despliegue. Railway facilitó el despliegue en la nube conectando automáticamente la base de datos PostgreSQL mediante variables de referencia.